

The bionicreading package

Explorer

2026-04-27

Abstract

bionicreading highlights the leading part of western ASCII words. It provides a small $\text{\LaTeX}$ 3 interface backed by LuaTeX node processing. Both inline commands and paragraph environments mark their contents with a LuaTeX attribute; the Lua back-end later converts marked western glyphs during paragraph processing. CJK glyphs and mathematical material are not converted.

1 Overview

The package implements a bionic-reading-style visual treatment for western text. For each ASCII word, a calculated prefix is switched to the bold version of the current font. The calculation follows the fixation tables used by the text-vide project: higher fixation-point values generally produce shorter highlighted prefixes.

The package is intentionally scoped to western ASCII words. It does not attempt to segment CJK text, and it does not modify math lists. LuaLaTeX is required because all conversion is performed on the node list after macro expansion.

2 User interface

<code>\bionicsetup</code>	<code>\bionicsetup{\textit{options}}</code>
---------------------------	---

Sets global package options. The currently supported keys are `enabled` and `fixation-point`.

<code>\bionic</code>	<code>\bionic[\textit{options}]{\textit{text}}</code>
----------------------	---

Marks inline or short text for bionic-reading conversion. The command does not scan the argument as a TeX string. It sets a LuaTeX attribute while the argument is typeset; the Lua backend later processes the marked glyph nodes.

`\bionicstart` `\bionicstart[<options>]`

`\bionicstop` `<text>`

`\bionicstop`

Enables and disables marking for an explicit scope. The start command registers the current normal font and its bold counterpart as a font pair, then marks subsequently created glyphs with the current fixation point.

`bionicpar` `\begin{bionicpar}[<options>]`

`<paragraphs>`

`\end{bionicpar}`

A paragraph environment for bionic reading. The environment uses the same LuaTeX attribute mechanism as `\bionic`, but it also terminates the current paragraph at the end of the environment.

3 Options

Key	Value	Meaning
enabled	boolean	Enables or disables conversion.
fixation-point	1–5	Selects the fixation table.

4 Examples

```
\usepackage{bionicreading}
```

```
\bionic{Confucius said: Madam, I'm Adam.}
```

```
\bionic[fixation-point=5]{text-wide and internationalization}
```

```
\begin{bionicpar}[fixation-point=3]
```

```
\lipsum[1]
```

```
\end{bionicpar}
```

Both interfaces use the same LuaTeX node backend. Use `\bionic` for inline spans and `bionicpar` for paragraph blocks.

5 LuaTeX backend

The Lua backend scans paragraph node lists in `pre_linebreak_filter`. Only glyphs carrying the package attribute are converted. ASCII letters and digits are accumulated as one word. CJK glyphs, spaces, punctuation, math nodes, and other non-word nodes

terminate the current word. Kerning nodes and discretionary hyphenation nodes are not word boundaries: TeX can insert them inside western words, and treating them as boundaries would restart prefix highlighting in the middle of a word.

The backend changes only glyph font IDs. It does not insert new nodes and does not alter the textual content. This keeps macro expansion, CJK text, and math material under TeX’s normal control.

6 Implementation

6.1 Package file

```

1 <*package>
2 \NeedsTeXFormat{LaTeX2e}
3 \ProvidesExplPackage
4   {bionicreading}
5   {2026-04-27}
6   {0.1.0}
7   {Bionic-reading style highlighting for ASCII western text}
8
9 \RequirePackage{expl3}
10 \RequirePackage{xparse}
11
12 \ExplSyntaxOn
13
14 \msg_new:nnn { bionicreading } { invalid-fixation-point }
15   {
16     Invalid~fixation-point~'#1'.~
17     The~value-must-be-an-integer-from-1-to-5.
18   }
19 \msg_new:nnn { bionicreading } { luatex-required }
20   { The~bionicreading~package~requires~LuaLaTeX. }
21
22 \sys_if_engine luatex:F
23   { \msg_fatal:nn { bionicreading } { luatex-required } }
24
25 \RequirePackage{luatexbase}
26
27 \bool_new:N \l__bionicreading_enabled_bool
28 \int_new:N \l__bionicreading_fixation_point_int
29 \int_new:N \l__bionicreading_normal_font_int
30 \int_new:N \l__bionicreading_bold_font_int
31 \newluatexattribute \g__bionicreading_attribute

```

```

32
33 \keys_define:nn { bionicreading }
34 {
35     enabled .bool_set:N = \l__bionicreading_enabled_bool,
36     enabled .initial:n = true,
37     fixation-point .code:n =
38     {
39         \int_compare:nNnTF {#1} < { 1 }
40         { \msg_error:nnn { bionicreading } { invalid-fixation-point } {#1} }
41         {
42             \int_compare:nNnTF {#1} > { 5 }
43             { \msg_error:nnn { bionicreading } { invalid-fixation-point } {#1} }
44             { \int_set:Nn \l__bionicreading_fixation_point_int {#1} }
45         }
46     },
47     fixation-point .initial:n = 1,
48 }
49
50 \NewDocumentCommand \bionicsetup { m }
51 { \keys_set:nn { bionicreading } {#1} }
52
53 \NewDocumentCommand \bionic { O{} +m }
54 {
55     \group_begin:
56     \keys_set:nn { bionicreading } {#1}
57     \__bionicreading_lua_register_font_pair:
58     \__bionicreading_set_attribute:
59     #2
60     \group_end:
61 }
62
63 \directlua
64 {
65     bionicreading = dofile(kpse.find_file("bionicreading.lua"))
66     bionicreading.set_attribute(
67         luatexbase.attributes["g__bionicreading_attribute"]
68     )
69     bionicreading.register_callbacks()
70 }
71
72 \cs_new_protected:Npn \__bionicreading_set_attribute:
73 {
74     \bool_if:NTF \l__bionicreading_enabled_bool

```

```

75     {
76         \setluatexattribute \g__bionicreading_attribute
77         { \int_use:N \l__bionicreading_fixation_point_int }
78     }
79     { \unsetluatexattribute \g__bionicreading_attribute }
80 }
81
82 \cs_new_protected:Npn \__bionicreading_lua_register_font_pair:
83 {
84     \group_begin:
85     \int_set:Nn \l__bionicreading_normal_font_int { \fontid \font }
86     \bfseries
87     \int_set:Nn \l__bionicreading_bold_font_int { \fontid \font }
88     \directlua
89     {
90         bionicreading.register_font_pair(
91             \int_use:N \l__bionicreading_normal_font_int,
92             \int_use:N \l__bionicreading_bold_font_int
93         )
94     }
95     \group_end:
96 }
97
98 \NewDocumentCommand \bionicstart { 0{ } }
99 {
100     \group_begin:
101     \keys_set:nn { bionicreading } { #1 }
102     \__bionicreading_lua_register_font_pair:
103     \__bionicreading_set_attribute:
104 }
105
106 \NewDocumentCommand \bionicstop { }
107 { \group_end: }
108
109 \NewDocumentEnvironment { bionicpar } { 0{ } }
110 { \bionicstart [#1] }
111 { \par \bionicstop }
112
113 \ExplSyntaxOff
114 \</package>

```

6.2 Lua backend

```

115 \<lua>

```

```

116 local M = {}
117
118 local glyph_id = node.id("glyph")
119 local math_id = node.id("math")
120 local disc_id = node.id("disc")
121 local kern_id = node.id("kern")
122
123 local boundaries = {
124   { 0, 4, 12, 17, 24, 29, 35, 42, 48 },
125   { 1, 2, 7, 10, 13, 14, 19, 22, 25, 28, 31, 34, 37, 40, 43, 46, 49 },
126   { 1, 2, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37, 39, 41, 43, 45, 47 },
127   { 0, 2, 4, 5, 6, 8, 9, 11, 14, 15, 17, 18, 20, 0, 21, 23, 24, 26, 27, 29, 30, 32, 33, 35, 37 },
128   { 0, 2, 3, 5, 6, 7, 8, 10, 11, 12, 14, 15, 17, 19, 20, 21, 23, 24, 25, 26, 28, 29, 30, 32, 34 },
129 }
130
131 M.attribute = nil
132 M.font_map = {}
133 M.callbacks_registered = false
134
135 local function is_ascii_letter(char)
136   return (char >= 65 and char <= 90) or (char >= 97 and char <= 122)
137 end
138
139 local function is_ascii_digit(char)
140   return char >= 48 and char <= 57
141 end
142
143 local function is_ascii_alnum(char)
144   return is_ascii_letter(char) or is_ascii_digit(char)
145 end
146
147 local function fixation_length(word_length, fixation_point)
148   local list = boundaries[fixation_point] or boundaries[1]
149   for index, boundary in ipairs(list) do
150     if word_length <= boundary then
151       local length = word_length - (index - 1)
152       if length < 0 then
153         return 0
154       end
155       return length
156     end
157   end
158   local length = word_length - #list

```

```

159     if length < 0 then
160         return 0
161     end
162     return length
163 end
164
165 local function node_marker(n)
166     if not M.attribute then
167         return nil
168     end
169     local marker = node.has_attribute(n, M.attribute)
170     if marker and marker >= 1 and marker <= 5 then
171         return marker
172     end
173     return nil
174 end
175
176 local function bold_word(word, has_letter, fixation_point)
177     if not has_letter or #word == 0 then
178         return
179     end
180     local bold_count = fixation_length(#word, fixation_point)
181     for i = 1, bold_count do
182         local glyph = word[i]
183         local bold_font = M.font_map[glyph.font]
184         if bold_font then
185             glyph.font = bold_font
186         end
187     end
188 end
189
190 function M.process_list(head)
191     if not M.attribute then
192         return head
193     end
194
195     local word = {}
196     local has_letter = false
197     local fixation_point = nil
198
199     local function flush()
200         if fixation_point then
201             bold_word(word, has_letter, fixation_point)

```

```

202     end
203     word = {}
204     has_letter = false
205     fixation_point = nil
206 end
207
208 for n in node.traverse(head) do
209     local marker = node_marker(n)
210     if n.id == glyph_id and marker and is_ascii_alnum(n.char) then
211         if fixation_point and fixation_point ~= marker then
212             flush()
213         end
214         fixation_point = marker
215         word[#word + 1] = n
216         if is_ascii_letter(n.char) then
217             has_letter = true
218         end
219     elseif n.id == kern_id or n.id == disc_id then
220         -- Font kerning and discretionary hyphenation can occur inside a word.
221         -- They must not restart fixation calculation for the following glyphs.
222     else
223         flush()
224     end
225 end
226
227 flush()
228 return head
229 end
230
231 function M.set_attribute(attribute)
232     M.attribute = tonumber(attribute)
233 end
234
235 function M.register_font_pair(normal_id, bold_id)
236     normal_id = tonumber(normal_id)
237     bold_id = tonumber(bold_id)
238     if normal_id and bold_id then
239         M.font_map[normal_id] = bold_id
240     end
241 end
242
243 function M.register_callbacks()
244     if M.callbacks_registered then

```



```

245     return
246 end
247 if luatexbase and luatexbase.add_to_callback then
248     luatexbase.add_to_callback("pre_linebreak_filter", M.process_list, "bionicreading")
249 else
250     callback.register("pre_linebreak_filter", M.process_list)
251 end
252 M.callbacks_registered = true
253 end
254
255 return M
256  $\langle$ /lua $\rangle$ 

```